

# BITENGINE: Query Engines Using Bitmap Indexing

Junchang Wang  
Nanjing University of Posts  
and Telecomm.  
wangjc@njupt.edu.cn

Fei Gao  
Nanjing University of Posts  
and Telecomm.  
1224045510@njupt.edu.cn

Fu Xiao  
Nanjing University of Posts  
and Telecomm.  
xiaof@njupt.edu.cn

Manos Athanassoulis  
Boston University  
mathan@bu.edu

## ABSTRACT

Bitmap indexes offer space and query efficiency for analytical workloads. Recent advances have extended bitmap indexes beyond read-only, low-cardinality attributes, but in DBMSs they remain largely confined to scan acceleration. In this paper, we extend the use of bitmap indexes by proposing BITENGINE, a novel *bitmap-oriented query engine* for analytical systems. Our design offers four key advantages compared to baseline designs. (1) Bitmap indexing significantly reduces the amount of data processed in scan-heavy operations (e.g., column scan and selection). (2) Bitmap indexing result bitvectors naturally map to modern AVX-512 mask registers, allowing computation-intensive operations (e.g., grouping and aggregation) to better leverage modern SIMD instructions. (3) BITENGINE transforms joins into predicate evaluations on bitmap-indexed foreign key columns, where bitmap indexes provide readily available query results. This eliminates the costly intermediate table generation in baseline designs, making BITENGINE orders of magnitude more space-efficient and immune to memory spills. (4) BITENGINE’s operators typically pass bitvectors that can be combined efficiently. This allows BITENGINE to decompose complex analytical queries (e.g., star joins) into independent subqueries and then merge their result bitvectors, significantly reducing intermediate data. BITENGINE can be integrated into modern analytical systems by replacing physical plan operators. Our evaluation integrating BITENGINE into DuckDB shows that BITENGINE achieves 1.4×–18.9× speedups on 12 out of 22 TPC-H queries.

## PVLDB Reference Format:

Junchang Wang, Fei Gao, Fu Xiao, and Manos Athanassoulis. BITENGINE: Query Engines Using Bitmap Indexing. PVLDB, XX(X): XX–XX, XXXX. doi:XX.XX/XXX.XX

## PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://github.com/junchangwang/BitEngine>.

## 1 INTRODUCTION

**Bitmap Indexing** is widely used in analytical DBMSs to improve query performance [4, 11, 19, 21, 42, 44, 46]. A bitmap index targets an attribute  $A$  with  $c$  distinct values. For each distinct value, it maintains an array of  $n$  bits, denoted *bitvector*. Conceptually, all bitvectors form a  $c \times n$  *bit matrix* [11, 43], in which the  $i^{th}$  bits of all bitvectors collectively encode the  $i^{th}$  entry of  $A$  using various

*encoding schemes*. In the basic *equality encoding* [12], the bit of the corresponding bitvector is set to 1, while all other bits in the same row are set to 0.

**Efficient Queries.** Bitmap indexes yield readily available results for analytical queries. Point queries (e.g., “ $A = x$ ”) are resolved by simply returning the bitvector corresponding to  $x$ . For range queries (e.g., “ $A \leq x$ ”), state-of-the-art *group encoding* [44] maintains sums of adjacent bitvector groups, thereby limiting query processing to merging a small number of cumulative bitvectors. Furthermore, the columnar storage structure of bitmaps ensures that the core operations (reading bitvectors and performing logical operations among them) exhibit strong spatial locality [15]. This efficiency is further amplified by modern hardware, including larger caches, improved hardware prefetching, and wider SIMD instructions.

**Challenges and Advancements.** Despite their advantages, bitmap indexes have long been limited to indexing read-only, low-cardinality attributes, due to prohibitive update and/or storage overheads [29, 40]. To address these limitations, UpBit [4] and CUBIT [42] introduce update-friendly bitmap indexes, and RABIT [44] extends these designs to efficiently support high-cardinality attributes. These advances, for the first time, enable bitmap indexing on attributes of any cardinality in tables ranging from read-only to frequently updated, with minimum memory overhead [42, 44].

**Motivation.** These advancements raise a fundamental question: **beyond traditionally acting as scan accelerators, to what extent can the state-of-the-art bitmap indexes be leveraged in DBMS query engines?** To address this, we introduce BITENGINE, a query engine that leverages state-of-the-art bitmap designs [42, 44] to accelerate core analytical operations.

**BITENGINE** introduces a suite of bitmap-oriented operators (termed *BitOperators*) that can replace standard operators in physical plans. Specifically, BITENGINE’s *Selection* operator retrieves and merges compressed bitvectors from bitmap indexes on table columns. Compared to leading baselines (scans on bit-packed columns and sparse indexes like Column Sketches [16]), BITENGINE reduces the amount of data read for selection by 1.9×–2.4× (§3.1). Its *Projection* operator, by leveraging the upstream selection result bitvector, selectively retrieves only the qualifying column segments. Unlike modern analytical systems that read predicate and projection columns simultaneously (using only min/max filters), BITENGINE consistently minimizes projection I/O across varying workloads (§3.2). BITENGINE’s *Aggregation* operator receives bitvectors from upstream operators (e.g., selection), which naturally map to AVX-512 mask registers, making Aggregation highly amenable to data parallelism on modern hardware (§3.3). BITENGINE’s *Grouping* operator calculates a result bitvector for each group, which is then used for AVX-512 accelerated aggregation. This design eliminates the maintenance of lightweight hash tables required in traditional grouping, outperforming the baseline by 1.6×–4.7× across all query selectivities

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing [info@vldb.org](mailto:info@vldb.org). Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. XX, No. XX ISSN 2150-8097.  
doi:XX.XX/XXX.XX

(§3.4). Its *Binary Join* operator transforms joins into predicates on fact table foreign keys, leveraging bitmap indexes to execute the predicates by merging bitvectors. This addresses the critical challenge of handling highly variable intermediate data in standard hash-based join operators, which can frequently exceed the volume of the original data [8, 25] and lead to unexpected memory spills [36, 41]. Our evaluation on TPC-H shows BITENGINE is up to 27× more space efficient and 2.6× faster than state-of-the-art hash joins (§3.5). BITENGINE executes *Star Joins* as independent binary joins, producing bitvectors that are subsequently bitwise ANDed to identify qualifying tuples passing all joins. Unlike traditional methods that process a star join as a sequence of binary joins, BITENGINE avoids generating massive, unpredictable intermediate data [36], achieving a 2× speedup on TPC-H Q5 while reducing intermediate data by an order of magnitude (§3.6).

**Integration into DBMSs.** BITENGINE leverages bitmap as secondary structures to accelerate independent DBMS operators without requiring modifications to their core query processing logic. We demonstrate this by integrating BITENGINE into DuckDB, a vectorized push-based analytical DBMS (§2.2). BITENGINE traverses generated physical plans bottom-up (from table scan to final output) to identify candidate operators. It replaces a standard operator only if the required bitmap indexes exist and the BitOperator has a lower estimated cost. All other operators remain unchanged, allowing a non-invasive integration into DuckDB (§4).

**Contributions.** The paper offers the following contributions.

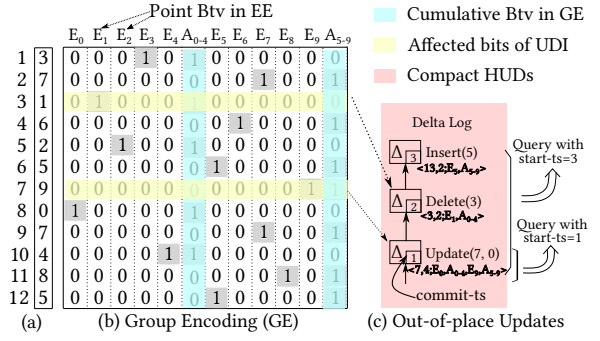
- Despite recent advances, bitmap indexes remain primarily used as scan accelerators. We present the first systematic research on using bitmap indexes at the core of analytical DBMSs, and introduce BITENGINE, the first bitmap-oriented query engine.
- We present BitOperators, a suite of bitmap-oriented DBMS operators. We highlight their distinct advantages, including reduced storage I/O and intermediate data size, improved SIMD utilization, and a better fit for complex analytical queries involving multi-column predicates, grouping, and star joins.
- We integrate BITENGINE into the production-grade DuckDB. Since BITENGINE accelerates operators by leveraging bitmap indexes, its design ensures non-invasive integration: for each query, it traverses the generated physical plan to replace feasible operators without altering the core query processing procedure.
- Our evaluation shows that BITENGINE outperforms baseline DuckDB by 1.4×–18.9× on 12 out of 22 TPC-H queries with known performance choke points. This demonstrates that *bitmap indexing can and should be an integral component of both indexing and query processing* of modern DBMSs.

## 2 BACKGROUND

We now present the necessary background on analytical DBMSs, bitmap indexes designed for frequently updated, high-cardinality attributes, and the SIMD instructions used in BITENGINE.

### 2.1 Bitmap Indexes

**Classical Bitmap Indexes.** The seminal bitmap indexing structure [46] associates a bitvector with each distinct value of an indexed attribute  $A$ . Subsequent research [26, 27, 47] evolves this design

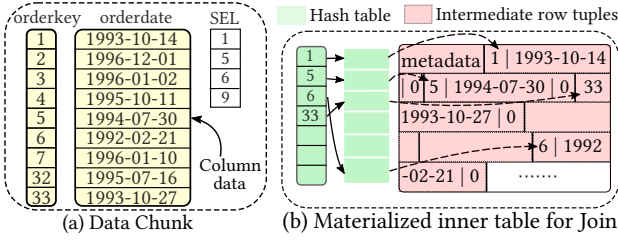


**Figure 1: (a) Attribute  $A$  with cardinality  $C = 10$ . (b) State-of-the-art Group Encoding (GE) extends Equality Encoding (EE) by additionally storing the logical sum of contiguous values. This maintains space efficiency after compression and facilitates range queries. (c) Updates are performed out-of-place by compactly storing the affected bit strings, along with timestamps, in a per-bitmap log. This ensures that concurrent queries and UDIs do not block one another.**

into a *bit-matrix* organization, combining per-bitvector columnar storage with conceptual row-wise bit strings that encode entries in  $A$  using various schemes. In the fundamental *equality encoding* (EE) [11], the index comprises  $C$  bitvectors, where  $C$  is the cardinality of  $A$ . The  $k^{th}$  bit of a bitvector  $B$  is set to 1 if the  $k^{th}$  row matches the value of  $B$ , and 0 otherwise. As an example, Figure 1a shows the projection on the TPC-H  $L\_quantity$  attribute (assuming cardinality = 10 for brevity), and the gray bitvectors labeled  $E_i$  (where  $i$  denote the value) in Figure 1b constitute the corresponding EE bitmap index. Alternative encoding schemes, such as *range encoding* (RE) and *interval encoding* (IE) [11, 12], encode additional bits per value to facilitate range queries. However, existing designs limit to read-only, low-cardinality attributes [4, 42, 44]. For example, while EE bitmap indexes are efficient for point queries (e.g., “ $A = x$ ”), performing range queries like “ $A \leq x$ ” on high-cardinality attributes requires merging a large number of bitvectors, incurring significant performance overhead.

**Group Encoding (GE).** Recent research [44] proposes a new encoding scheme called *Group Encoding* (GE). GE maintains the bitvectors in EE (referred to as *point bitvectors*) for fast point queries. Additionally, it introduces *cumulative bitvectors*, each of which stores the logical sum of a group of contiguous values of the indexed attribute, created by performing bitwise ORs on their respective point bitvectors. The blue bitvectors labeled  $A_{i-j}$  in Figure 1b represent the cumulative bitvectors that aggregate values in the range  $[i, j]$ . GE overcomes existing bitmap indexes’ challenges as follows. (1) The major body of GE bitmap indexes (i.e., point bitvectors) remains highly compressible, making GE as space-efficient as EE. (2) Each *Update*, *Delete*, and *Insert* operations (hereafter, *UDIs*) flips at most four bits, making efficient updates possible. (3) Range queries are answered by summing a few cumulative bitvectors and point bitvectors from boundary groups, addressing the costly merging overhead on high-cardinality attributes.

**Update-friendliness.** Recent studies [4, 42, 44] address the updatability challenge by proposing an out-of-place update strategy. For example, the state-of-the-art RABIT records the horizontal bitwise



**Figure 2: (a) The baseline vectorized columnar DBMS (e.g., DuckDB) passes Data Chunks between operators. (b) The Join operator materializes the inner table, by assembling the column values in Data Chunks into row tuples to reduce cache misses during probing.**

representation of updated bits for each UDI (termed HUD) in a per-bitmap Delta Log, enabling efficient atomic updates without locking entire bitvectors. RABIT stores each update as  $\langle \text{row\_id}, n\_ones, p1, p2, \dots \rangle$ , where  $n\_ones$  indicates the number of bits set to 1 in the HUD, followed by their positions. As illustrated in Figure 1c, an update operation changing the 7<sup>th</sup> row to value 0 appends a HUD  $\langle 7, 4; E_0, A_0-4, E_9, A_5-9 \rangle$  to Delta Log. Similarly, a delete operation removing the 3<sup>rd</sup> row, and an insertion of value 5 are performed by appending their HUDs at the tail of Delta Log. Furthermore, RABIT introduces a lightweight snapshotting mechanism with timestamping. By retrieving qualifying HUDs based on their arrival timestamps and merging them into the base bit-matrix, queries and UDIs operate on different index snapshots, ensuring that concurrent operations do not block one another. As illustrated in Figure 1c, queries arrived at timestamps 1 and 3 retrieve different HUD sets and thus obtain different snapshots. HUDs are merged back into bitvectors in batches by dedicated background maintenance threads. By generating multiple versions of the affected bitvector segments, these merge operations are transparent and do not block queries or UDIs [44].

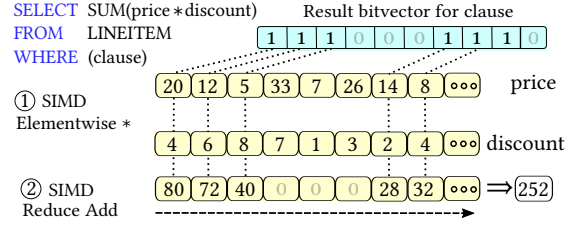
BITENGINE adopts state-of-the-art bitmap designs [42, 44] that support efficient point and range queries on attributes of any cardinality (referred to as *bitmaps* in this paper).

## 2.2 Baseline DBMS Execution Model

We choose DuckDB [32] as our DBMS baseline and follow its terminology throughout this paper.

**Storage.** Modern analytical DBMSs (e.g., DuckDB [32] and ClickHouse [34]) organize storage data in a Parquet-like layout [3]. For example, DuckDB horizontally slices and organizes tuples into *Row Groups*, each containing 120K rows by default. Within each Row Group, data is stored in a columnar format as value vectors, on which compression is applied [39] and column-level statistics like zone maps are maintained.

**Vectorized Execution.** DuckDB employs vectorized execution [10]. The referenced columns of a query are first fetched from Row Groups, decompressed into 32- or 64-bit values, and partitioned into *Data Chunks* of 2,048 values. Each data chunk is associated with a *selection vector* (SEL), which stores the local row IDs of the qualifying tuples within each data chunk, as illustrated in Figure 2a. In DuckDB, Data Chunks are the fundamental execution unit passed between operators: their value vectors are immutable, while SEL are updated by operators (e.g., Selection in §3.1).



**Figure 3: BITENGINE utilizes AVX-512 by loading selection result bitvectors into mask registers to execute (1) projection and (2) aggregation in 512-bit batches.**

**Materialization** is only performed for joins in modern analytical DBMSs [2, 23, 36]. It assembles row tuples of the inner table to store payloads sequentially, reducing random memory access during probing. In DuckDB, the inner table’s *Data Chunks* are fetched and explicitly converted to row records that are written sequentially into a *Data Table*. A hash table maps each row key to the starting address of the corresponding record in the *Data Table*, as illustrated in Figure 2b. Materializing the inner table allows the join to retrieve a tuple with a single memory access during probing, significantly reducing cache misses.

**Pipeline Execution.** DuckDB breaks a physical query plan into *pipelines*: chains of operators that execute in parallel and terminate at the first operator that must wait for all input. Each pipeline begins with a *source* operator (e.g., column scan) that fetches Data Chunks from storage or upstream pipelines. Data then flows through the operator chain (e.g., Projection) at the granularity of Data Chunks to maximize CPU cache efficiency. Finally, a *sink* operator (e.g., Join and Grouping) consumes and materializes input data, acting as the synchronization point to prepare output for the next pipeline.

## 2.3 Advancing SIMD

Columnar DBMSs extensively leverage SIMD for vector processing [22, 31, 45]. This paper focuses on the Advanced Vector Extensions (AVX-512), supported by modern server CPUs, including Intel Xeon and AMD EPYC. AVX-512 improves not only SIMD register width (up to 512 bits) but, more importantly, programming flexibility via per-element masking that allows instructions to be applied selectively to specific packed values. Take the query in Figure 3 as an example. The selection result bitvector maps directly to mask registers, while column data to operand registers, naturally enabling element-wise multiplication on qualifying values and aggregation in batches (e.g., sixteen 32-bit integers).

## 3 BITOPERATORS IN BITENGINE

We now discuss the fundamental BitOperators: Selection (§3.1), Projection (§3.2), and Aggregation (§3.3), by examining the following TPC-H Q6 variant, deferring Grouping and Join to §3.4–§3.5. In this section, we use the term “baseline DBMS” to refer to DuckDB. Our testbed server has native AVX-512 support, and we evaluate using TPC-H at scale factor 10 (see details in §6).

```
SELECT SUM(l_price * l_discount) FROM LINEITEM
WHERE l_quantity < Q1 and l_shipdate in S1;
```

### 3.1 Selection Operator

The Selection operator reads required columns, filters rows using query predicates, and outputs a collection of RIDs (for the baseline DBMS) or a result bitvector (for BITENGINE).

**Baseline DBMS** performs a *Sequential Scan* on the most selective column ( $l\_shipdate$  in our example), utilizing predicate pushdown (min/max filters) to skip data chunks. Qualifying RIDs are collected in the SELs of Data Chunks and subsequently used to probe remaining columns ( $l\_quantity$ ) to refine selection results. The final SELs, which contain the RIDs of all rows satisfying all predicates, are forwarded to subsequent operators. Analytical DBMSs can avoid scanning predicate columns when query selectivity is low [17] by using *Sparse Indexes* [16, 37] that have a smaller memory footprint than tree-based indexes. We use Column Sketches (1-byte sketch size) as a representative design and implement the associated optimizations [16], in which sketches are evaluated using AVX-512 instructions before accessing each chunk (2,048 values). Because Column Sketches do not natively support multi-column sketches on numerical attributes (Appendix C of [16]), we evaluate the sketches for each of the predicate columns separately and then efficiently merge the resulting ID lists using logical bitwise ANDs.

**BITENGINE** maintains bitmap indexes on the predicate columns in the fact table LINEITEM, because they are update-friendly and applicable to high-cardinality attributes. In our example, we build bitmap indexes for the attributes  $l\_shipdate$  (card. = 2,526 and GE group size = 3 months) and  $l\_quantity$  (card. = 50 and group size = 5). The query predicate is evaluated by performing logical bitwise AND/ORs on ~8 bitvectors (4 out of 28 cumulative bitvectors for  $l\_shipdate$  and 4 cumulative bitvectors along with a few point bitvectors for  $l\_quantity$ ). The result bitvector is then forwarded to subsequent operators. Our evaluation shows that BITENGINE’s selection offers the following advantages.

**BITENGINE Reads Less Data.** To evaluate the memory efficiency of various selection approaches, we introduce a *accessed data volume model*, following the notation in Table 1. We assume a column contains  $N$  values and the indexed attribute has a cardinality of  $C$ . Since values are typically bit-packed, we denote the size of each value as  $S_{bp}$ , which is typically 8-16 bits [45]. For Column Sketches,  $S_{sk}$  represents the per-value sketch size (1 Byte in our implementation), and  $r_{cc}$  is the candidate check ratio. Finally, for BITENGINE that utilizes Group Encoding-based bitmaps,  $G$  denotes the group size, and  $S_u$  and  $S_c$  represent the sizes of uncompressed and compressed bitvectors, respectively.

A sequential scan traverses the two predicate columns, resulting in an accessed data volume of  $S_{bp} \cdot N$  per column. For queries using Column Sketches, we assume the optimal scenario where no candidate checks are executed, and all qualifying tuples are correctly identified by the sketches (i.e.,  $r_{cc} = 0$ ). Under this assumption, the estimated data accessed volume is simply  $S_{sk} \cdot N$ . A point query on BITENGINE requires only a single point bitvector lookup, incurring a read cost of  $S_c$ . For a range predicate on BITENGINE, the upper bound of the data accessed volume can be estimated by the cost of accessing half of the cumulative bitvectors ( $\lfloor C/2G \rfloor \cdot S_u$ ) and half of the point bitvectors in a group ( $\lfloor G/2 \rfloor \cdot S_c$ ). Consequently, the average data accessed volume for a range query is given by  $\lfloor C/4G \rfloor \cdot S_u + \lfloor G/4 \rfloor \cdot S_c$ , which is calculated as follows.

Table 1: Notations in data read volume model.

|                 |                                     |                                                                                                                                                                |
|-----------------|-------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Data            | $N$<br>$C$                          | Number of values<br>Cardinality of the indexed attribute                                                                                                       |
| Scan            | $S_{bp}$                            | Bit-packed size per value                                                                                                                                      |
| Column Sketches | $S_{sk}$<br>$r_{cc}$                | Per-value sketch size (1 Byte)<br>Candidate check ratio                                                                                                        |
| BITENGINE       | $G$<br>$S_u$<br>$r_{comp}$<br>$S_c$ | Group size of GE<br>Size of uncompressed bitvector ( $N/8$ )<br>Compression ratio of point bitvectors<br>Size of compressed point bitvector ( $S_u/r_{comp}$ ) |

$$S_{BitEngine} = \left\lfloor \frac{C \cdot S_u}{4 \cdot G} \right\rfloor + \left\lfloor \frac{G \cdot S_c}{4} \right\rfloor \approx \left\lfloor \frac{C}{4 \cdot G} + \frac{G}{4 \cdot r_{comp}} \right\rfloor \cdot S_u \quad (1)$$

Prior work [44] shows that for uniformly distributed datasets, the compression ratio of a bitvector under WAH compression with 32-bit words can be approximated as  $r_{comp} \approx C \cdot \alpha$ . Furthermore, when the attribute cardinality  $C$  exceeds 50, then  $\alpha \approx 0.016$ , consistent with the results reported by the WAH authors [48]. In our implementation, we set the group size  $G$  to approximately 1/16 of the cardinality  $C$ , which aligns with the optimal practice for Group Encoding [44]. Therefore, the average data accessed volume for BITENGINE can be expressed as:

$$S_{BitEngine} \approx \left( \frac{C}{4 \cdot G} + \frac{G}{4 \cdot \alpha \cdot C} \right) \cdot S_u \approx \left( \frac{16}{4} + \frac{1}{4 \cdot 0.016 \cdot 16} \right) \cdot S_u \approx 5 \cdot S_u \quad (2)$$

This theoretical model predicts that BITENGINE’s Selection achieves a data accessed volume of  $2 \cdot 5 \cdot N$ , which is 1.6× and 3.2× smaller than that of Column Sketches and sequential scans, respectively. Our evaluation confirms that in our example query, BITENGINE reads 188MB of data, approximately 1.9× and 2.4× smaller than the baselines, as shown in Table 2.

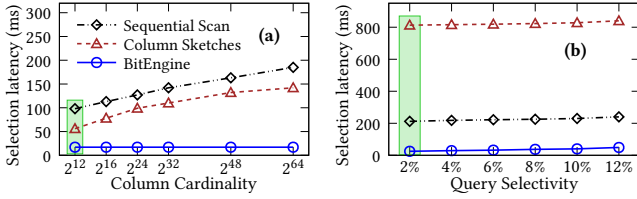
Table 2: Bytes read by different selection strategies.

|           | Notes               | Theoretical                  | Evaluation | Inc.  |
|-----------|---------------------|------------------------------|------------|-------|
| Scan      | Bit-packed values   | $2 \cdot S_{bp} \cdot N$     | 452MB      | +2.4× |
| Sketches  | One-byte sketches   | $2 \cdot 8 \cdot N \uparrow$ | 360MB      | +1.9× |
| BITENGINE | With group encoding | $2 \cdot 5 \cdot N$          | 188MB      | –     |

**BITENGINE Provides Fast, Robust Queries.** Bitmap indexing results are readily available, leading to faster performance and reduced sensitivity to predicate parameters. We demonstrate this by varying two key selection parameters (column cardinality and query selectivity) and comparing single-column and multi-column predicates using the example query.

*Insensitive to Column Cardinality.* We first evaluate only the predicate  $l\_quantity < Q1$  and vary the attribute cardinality from  $2^{12}$  to  $2^{64}$  (implying bit-packed storage sizes from 12 to 64 bits). By adjusting  $Q1$ , we fix query selectivity to 2%. The GE group size in the bitmap index is set to 1/16 of the cardinality. As shown in Figure 4a, as cardinality increases, the query latency of sequential scan increases sharply, because of the larger amount of data read. The query latency for Column Sketches also increases due to the higher number of candidate checks resulting from the increased cardinality. In contrast, BITENGINE’s performance is almost insensitive to cardinality because it primarily merges a few cumulative bitvectors.





**Figure 4: BitENGINE outperforms selection baselines and is insensitive to (a) column cardinality and (b) query selectivity, for (a) single- and (b) multi-column predicates (green boxes).**

For attributes with 64-bit values, BITENGINE is 2× and 3.4× faster than Column Sketches and scans with bit-packing, respectively.

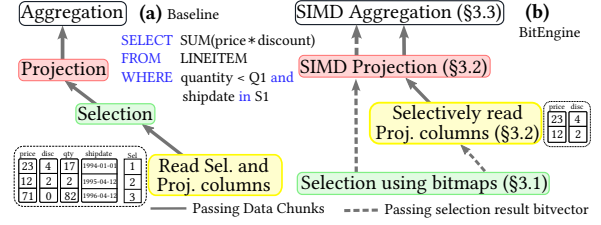
**Better Support for Multi-Column Predicates.** We now fix the value size to 12 bits, and additionally measure the second predicate on  $l\_shipdate$  to evaluate the performance of multi-column predicates. The green-boxed results in Figures 4a and 4b are comparable, except that the latter applies the predicate across both columns. We make the following observations. (1) Column Sketches lead to substantial performance degradation because the selection operator must merge results from two independent sketches. Both directly merging RID lists and first converting them to bitvectors incur significant overhead. (2) Operators using sequential scans must maintain the result RID list and probe subsequent columns, introducing data-dependent branches. Prior work notes that such branches, common in selections with moderate selectivity, are hard to predict and can significantly slow query performance [10]. (3) In contrast, BITENGINE’s Selection performs consistently for both single-column and multi-column predicates, because it simply fetches bitvectors from different bitmap indexes and merges them with bitwise logical operations. In our example query, BITENGINE achieves 25ms selection latency, 8.5× faster than the sequential scan and 32.5× faster than Column Sketches in the baseline DBMS.

**Scales with Query Selectivity.** Traditional secondary indexes (e.g., B-Trees) are widely acknowledged to outperform sequential scans only at very low selectivities (e.g., <2%), with performance degrading rapidly as selectivity increases [17, 35, 42]. The key reasons are (1) cache misses during B-Tree leaf nodes traversal and (2) their *non-clustered* storage layout [38], which requires sorting the resulting ID list before accessing storage data or performing out-of-order data accesses. In contrast, BITENGINE’s Selection avoids these indexing penalties because its core operations (sequentially reading bitvectors) increasingly benefit from larger caches and stronger hardware prefetching on modern CPUs to minimize cache misses. Furthermore, its Group Encoding scheme significantly reduces bitvector merging overhead as selectivity rises. Consequently, BITENGINE scales with query selectivity, as shown in Figure 4b. *While modern analytical DBMSs (e.g., DuckDB) often abandon secondary indexes in favor of sequential scans, BITENGINE supports a broad selectivity range, making secondary indexing viable again for these systems.*

### 3.2 Projection Operator

The Projection operator retrieves the columns specified in the SELECT clause and evaluates element-wise expressions on the qualifying tuples (e.g., multiplication in our example).

**Baseline DBMS** typically employs one of the two approaches to retrieve projection columns. (1) It performs selection on predicate



**Figure 5: In columnar DBMSs using Parquet-based storage, (a) the baseline reads all predicate and projection columns, then performs selection, projection, and aggregation operations on RIDs stored in each Data Chunk’s SEL. In contrast, (2) BitENGINE first retrieves selection result bitvectors using bitmap indexing, enabling selective retrieval of projection columns, SIMD projection and aggregation operations.**

columns ( $l\_quantity$  and  $l\_shipdate$  in our example), before selectivity retrieving projection columns ( $l\_price$  and  $l\_discount$ ). (2) It retrieves both predicate and projection columns simultaneously, utilizing predicate pushdown (e.g., zone maps) to skip Data Chunks prior to selection. While the first approach minimizes I/Os and fits pure columnar stores (e.g., MonetDB) for highly selective queries, the second approach maximizes sequential I/Os for modern analytical DBMSs that leverage Parquet-based storage and vectorized engines. Our evaluation confirms that the second approach outperforms the first once query selectivity exceeds 1% in our testbed; we therefore adopt the second approach as our projection baseline. Consequently, Data Chunks transferred across operators contain both predicate and projection columns, along with qualifying RIDs stored in SELs, as illustrated in Figure 5a. Using these Data Chunks, the baseline iterates over the qualifying tuples and executes the projection operation (multiplication) element-wise.

**BITENGINE** performs selection via bitmap indexing, eliminating the need to scan predicate columns. With the resulting selection bitvector, BITENGINE only retrieves the qualifying segments of the projection columns, as illustrated in Figure 5.

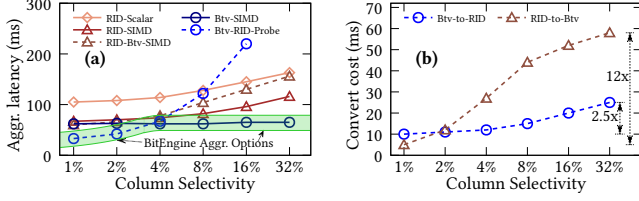
**Benefits.** BITENGINE enables efficient selective data loading for Parquet-based storage by using the preceding selection result bitvector. Unlike the baseline, which relies on zone maps, BITENGINE ensures the precise retrieval of qualifying segments of the projection columns irrespective of workload statistics, a critical advantage in real workloads (e.g., TPC-H with uniform distributions).

### 3.3 Aggregation Operator

The **Aggregation** operator processes qualifying column data to generate summarized results via aggregation functions such as SUM, MIN/MAX, and AVG, which appear in nearly all TPC-H queries.

**Baseline DBMS.** Upon receiving an RID list from the preceding selection operator, the baseline DBMS employs the following scalar approach, termed *RID-Scalar*, for aggregation: (1) Scan the  $l\_price$  and  $l\_discount$  columns, extracting qualifying values based on the RID list into two vectors. (2) Iterate over these two vectors, multiply each pair of values, and store the products in an intermediate result vector. (3) Traverse the result vector and compute the aggregation.

We further accelerate RID-scalar using modern SIMD instructions (see Algorithm 1). This new approach, termed *RID-SIMD*, uses RID lists as input. It employs AVX-512 *Selective Gather* instructions (Line 5) to retrieve qualifying values from column data, indexed by



**Figure 6: (a) With RID lists, baseline aggregation is slow, even though SIMD is used. Conversely, BITENGINE leverages incoming selection bitvectors to offer consistently fast SIMD aggregation. (b) While converting bitvectors to RIDs is efficient, the reverse is not. This allows BITENGINE to convert received bitvectors into RID lists to probe column data under low selectivity (green box). The x-axis is in log scale.**

8 64-bit RIDs from the RID list, and store the values contiguously in an output vector. Subsequently, we perform element-wise multiplication of the two vectors (Line 14) and compute their sum (Line 15) using AVX-512 instructions.

```

1 // Gather values from col[] using indexes in rid[]
2 gather_with_RID_SIMD(col[], rid[]):
3   for (each 8 RIDs batch in rid[]):
4     // Gathers 64-bit values from memory using 32-bit RIDs
5     values[] += _mm512_i32gather_epi64(col[], batch)
6   return values[]
7
8 price[] := gather_with_RID_SIMD(price, rid[])
9 disc[] := gather_with_RID_SIMD(discount, rid[])
10
11 // Aggregate Q6 over the two vectors of gathered values
12 aggregate_Q6_RID_SIMD(price[], disc[])
13   for (each 8-value batch in two vectors)
14     ret = _mm512_mul_epi64(price[batch], disc[batch])
15     sum += _mm512_reduce_add_epi64(ret) // Sum up 8 values
16   return sum

```

**Algorithm 1: SIMD aggr. using RIDs (RID-SIMD).**

BITENGINE receives a selection result bitvector that naturally maps to AVX-512 mask registers (§2.3). BITENGINE’s Aggregation operator gathers column values corresponding to the set bits in the bitvector and performs element-wise multiplication in a single instruction (Line 4 of Algorithm 2). Subsequently, it sums the results in batches using SIMD instructions (Line 5).

```

1 // Aggregate on two columns using selection bitvector Btv
2 aggregate_Q6_Btv_SIMD(price, disc, Btv)
3   for (each 16-value batch in columns):
4     _mm512i ret = _mm512_mask_mullo_epi32(price[batch], disc[batch],
5     Btv[batch])
6     sum += _mm512_reduce_add_epi32(ret) // Sum up 16 values
7   return sum

```

**Algorithm 2: SIMD aggr. using bitvector (Btv-SIMD).**

**Observations.** We evaluate the three aggregation approaches by varying query selectivity. The results are shown in Figure 6a (the three solid lines). We make the following observations.

**SIMD outperforms Scalar.** Both SIMD implementations outperform the scalar version because of their data-level parallelism. For example, RID-SIMD and Btv-SIMD are 1.4× and 2.5× faster than RID-Scalar for 32% selectivity, highlighting the importance of explicit SIMD coding for Aggregation.

**Btv-based SIMD Outperforms RID-based Version.** This performance benefit becomes more noticeable as selectivity increases. For example, Btv-SIMD slightly outperforms RID-SIMD at 1% selectivity, and becomes 2.4× faster as selectivity increases to 32%. This is mainly because of the following reasons. (1) RID-SIMD gathers qualifying values into two intermediate vectors before performing element-wise multiplication (Alg. 1, Lines 8-9). In contrast, Btv-SIMD directly maps the selection result bitvector into AVX-512 mask registers. This allows it to execute gather and multiplication operations in a single SIMD instruction (Alg. 2, Line 4), thus eliminating the need for intermediate vectors. (2) RID-SIMD iterates over an RID list (Alg. 1, Line 3) that grows directly with query selectivity. Conversely, BITENGINE leverages AVX-512 mask instructions to iterate over the selection result bitvector, which is fixed in size regardless of query selectivity. This enables BITENGINE to achieve stable performance across different selectivity levels.

```

1 // Convert RID list to bitvector
2 convert_RID_to_btv(RIDs[], btw[])
3   for (each ID in RIDs[]):
4     word = ID >> 6 // Find the matching 64-bit word
5     mask = 1 << (ID & 0x3F) // Offset in the word
6     btw[word] |= mask
7
8 // Convert bitvector to RID list
9 convert_btv_to_RID(btv[])
10   base_index := 0
11   for (each 32-bit batch in btw[]):
12     if (popcnt(batch) == 0): continue
13     // Extract the IDs corresponding to the 1s in batch
14     tmp[] = _mm512_maskz_compress(bits, [0, 1, 2, ..., 15])
15     // Add starting index to qualifying IDs
16     RID[] += _mm512_add_epi32(tmp[], base_idx)
17     base_index += 16
18   return RID[]

```

**Algorithm 3: Converting RIDs to bitvector, and vice versa.**

**Converting between RID lists and Bitvectors.** These findings raise a fundamental question: *Can DBMSs efficiently convert between a RID list and a bitvector when necessary?* To answer this, we implement both conversion algorithms. As illustrated in Algorithm 3, converting RID lists to bitvectors is mainly scalar-based. Applying SIMD to this function is complex and significantly degrades performance because contiguous RIDs often map to different memory words (Alg. 3, Lines 4–6), thus breaking SIMD instruction efficiency. In contrast, the reverse conversion fully leverages AVX-512 instruction. It extracts qualifying RIDs from set bits in the bitvector and adds the appropriate starting index to form the final RIDs (Lines 14–16), in 16-value batches.

Based on the conversion algorithms, we introduce two additional aggregation approaches. **RID-Btv-SIMD** adapts the baseline DBMS by converting received RID lists to bitvectors before the Btv-SIMD procedure. However, our evaluation shows that this approach becomes inefficient as query selectivity rises, performing up to 1.4× slower than the original RID-SIMD (red dashed lines in Figure 6a). This degradation stems from the conversion overhead; increasing selectivity from 2% to 32% increases the RID-to-bitvector cost by 12×, compared to only a 2.5× increase for the bitvector-to-RID conversion (Figure 6b). Conversely, **Btv-RID-Probe** first converts the received bitvector into an RID list, which is then used to probe column data for aggregation. As illustrated in Figure 6a, Btv-RID-Scalar

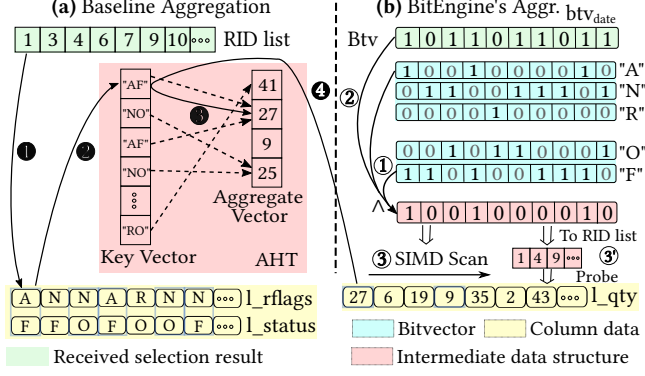


Figure 7: (a) DBMSs typically perform grouping by constructing a lightweight hash table over group-by columns ( $l\_rflags$  and  $l\_status$ ), and then scanning aggregation columns ( $l\_qty$ ) to update group states mapped via the hash table. (b) In contrast, BITENGINE leverages bitmap indexes on group-by columns to produce a result bitvector for each group, avoiding the hash table overhead. This bitvector is used to SIMD scan the aggregation column, or is converted to an RID list for probing for low selectivity queries (§3.3).

outperforms alternatives at low selectivity (e.g., < 1%) because it skips reading qualifying column segments.

BITENGINE's Aggregation Operator thus leverages efficient bitvector-to-RID conversion to dynamically switch between Btv-SIMD and Btv-RID-Probe depending on query selectivity, as illustrated in Figure 6a (green box). The selectivity is calculated using the received selection result bitvector (ratio of set bits), allowing the query engine to always identify the optimal strategy.

### 3.4 Grouping

The *Grouping* operator partitions qualifying rows into groups for fine-grained aggregation. We use TPC-H Q1 as an illustrative example, which generates a summary of items based on their fulfillment category (e.g., billed, shipped, and returned).

```
SELECT SUM(l_qty) FROM LINEITEM WHERE l_date < D1
GROUP BY l_rflag, l_status;
```

The Baseline DBMS typically leverages lightweight hash tables for grouping. As illustrated in Figure 7a, upon receiving the selection result (as a RID list) from the preceding selection operator ( $l\_date < D1$ ), the baseline processes as follows: ① Scans the group-by columns ( $l\_rflag$  and  $l\_status$ ). ② Computes composite keys for qualifying tuples to populate a Key Vector corresponding to the RID list. ③ Initializes an Aggregate Vector to store group states and maps Key Vector entries to their corresponding aggregates. For example, the 1<sup>st</sup> and 3<sup>rd</sup> entries (group "AF") map to the second slot in Aggregate Vector. ④ Scans the aggregation column ( $l\_qty$ ) and updates Aggregate Vector states referenced by Key Vector.

BITENGINE leverages bitmaps on group-by columns to compute a per-group result bitvector. This bitvector identifies qualifying tuples for each group and facilitates subsequent aggregations (§3.3), thereby avoiding the hash table overhead inherent in the baseline DBMS. In our example, two bitmaps are maintained for the fact table attributes  $l\_rflag$  and  $l\_status$ , respectively. To process the query, ① BITENGINE's Grouping operator enumerates each group and

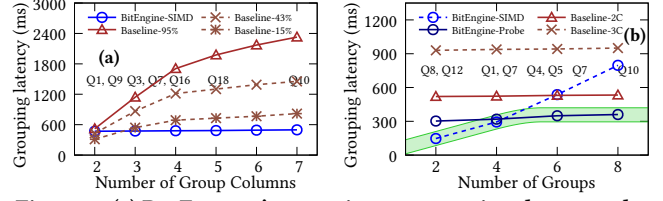


Figure 8: (a) BITENGINE's grouping operator is robust to selectivity and the number of group-by columns. (b) Leveraging the result bitvector for each group, BITENGINE either SIMD scans the column data or converts it to an RID list for probing, consistently outperforming the baselines (green box).

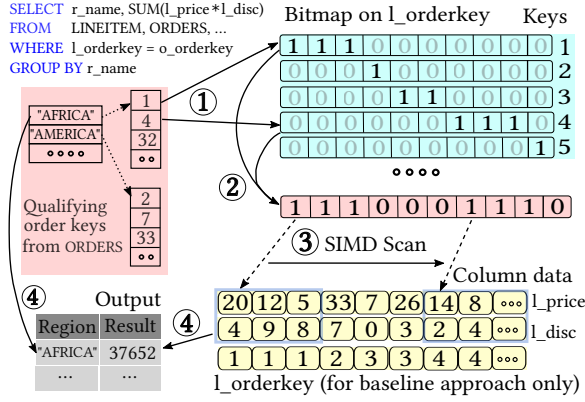
bitwise ANDs a bitvector pair ( $r, s$ ) from the two bitmap instances. For example, the bitvector for group "AF" is derived by ANDing the bitvectors for values "A" and "F" from their respective bitmaps, as illustrated in Figure 7b. ② It then bitwise ANDs this result with the selection result bitvector  $bv_{date}$  (from  $l\_date < D1$ ), yielding the final per-group bitvector  $bv_{group}$ . ③ BITENGINE then employs two aggregation strategies, inspired by the findings in §3.3. (1) It scans aggregation columns using AVX-512 mask instructions by directly mapping the column data ( $l\_qty$ ) and  $bv_{group}$  to operand and mask registers. This design, denoted as BITENGINE-SIMD, is optimal when the number of groups is small. (2) As group cardinality increases, BITENGINE first converts the resulting bitvectors to RID lists, then probes the column data, eliminating the overhead of scanning the column data for each group. We denote this strategy as BITENGINE-Probe.

**Observations.** We evaluate different grouping approaches by varying the number of columns, groups, and query selectivity. Our observations are as follows.

**BITENGINE is Robust to Multi-Column Grouping.** Complex analytical queries often involve grouping by multiple attributes. As shown in Figure 8a, six TPC-H queries utilize more than two group-by columns, with Q10 notably requiring seven. However, the baseline DBMS's grouping performance degrades in these scenarios. During hash table construction, the operator either scans all group-by columns simultaneously to compute composite keys in a single pass, or sequentially scans these columns to update the Key Vector. Our evaluation shows both approaches suffer from increased cache misses as the number of group-by columns rises. For example, increasing group-by columns from two to seven results in a 4.5 $\times$  grouping latency increase (see Baseline-95% in Figure 8a, where 95 indicates query selectivity discussed below). In contrast, BITENGINE's Grouping operator incurs negligible overhead because adding  $N$  group-by columns merely requires  $N$  additional bitwise AND operations, which mainly involve sequential memory access. Consequently, BITENGINE remains stable and becomes 4.7 $\times$  faster than the baseline for queries with seven group-by columns.

**BITENGINE is Robust to Selectivity.** The baseline approach is highly sensitive to query selectivity, because with high selectivity (e.g., ~95% in TPC-H Q1), it must maintain Key Vectors that are comparable in size to the raw data. To evaluate this, we lower the query selectivity from 95% to 15% using the predicate on  $l\_date$ . Figure 8a shows that while the baseline improves at lower selectivities due to reduced hash table maintenance overhead, BITENGINE provides stable performance and outperforms the baseline by 1.6 $\times$ –4.7 $\times$  across all selectivities.





**Figure 9: BITENGINE transforms joins into predicates on the fact table LINEITEM’s foreign key columns ( $l\_orderkey$ ). Upon receiving the qualifying keys per group by joining preceding dimension tables, BITENGINE (1) reads the corresponding bitvectors on  $l\_orderkey$ , (2) ORs them to generate a result bitvector for each group, (3) executes SIMD aggregates, and (4) propagates output row-wise.**

**Impact of Number of Groups.** BITENGINE-SIMD is optimal for small grouping cardinalities. Even against the baseline’s optimal configurations (two-column and three-column groupings, denoted as *Baseline-2C* and *Baseline-3C*), BITENGINE-SIMD outperforms them by 3.5× and 6.2×, respectively, for queries yielding two groups. While BITENGINE-SIMD degrades at larger grouping cardinalities, BITENGINE-Probe experiences only minor performance drops, consistently outperforming the baselines. BITENGINE therefore chooses between BITENGINE-SIMD and BITENGINE-Probe based on the grouping cardinality, as illustrated in Figure 8b (green box).

Note that in analytical queries (e.g., TPC-H), the grouping cardinality is typically low due to the following reasons. (1) Certain group combinations are inherently impossible. For example, in Q1, the combination of  $l\_rflag$  (3 values) and  $l\_status$  (2 values) generates 6 groups in theory. However, if a lineitem is *Order Open* (not yet shipped),  $l\_rflag$  can only be *Not Returned*, reducing the actual group cardinality to 4, lowering BITENGINE’s grouping overhead by 33%. (2) Most TPC-H query predicates restrict the valid values of group-by columns. For example, in Q12, although  $l\_shipmode$  has seven possible values, the WHERE clause restricts this down to two, substantially reducing the grouping overhead. As Figure 8b illustrates, the actual group cardinality for queries such as Q1, Q4, Q5, Q7, Q8, and Q10 is fewer than a dozen. Furthermore, for queries with an exceptionally large number of groups (e.g., Q16 with 28K and Q20 with 2K when SF = 10), grouping is not the performance bottleneck and the choice of grouping operator does not affect overall query performance, making BITENGINE’s Grouping the default choice for analytical queries.

### 3.5 Binary Join

The Join operator is ubiquitous in analytical workloads, appearing in 20 out of 22 TPC-H queries. Since the vast majority (18 of these 20) involve joining fact and dimension tables followed by aggregation, we focus on this primary scenario in this paper. The remaining cases (Q2 and Q20) involve lightweight, non-aggregating

joins on dimension tables that do not create performance bottlenecks; Therefore, BITENGINE processes them using the standard join operator in DuckDB.

```
SELECT r_name, SUM(l_price * l_disc)
FROM LINEITEM JOIN ORDERS JOIN ... JOIN REGION
WHERE l_orderkey in S1 GROUP BY r_name;
```

We use a TPC-H Q5 variant as the example query that performs a chain join between the LINEITEM fact table and ORDERS, CUSTOMER, NATION, and REGION. The preceding non-aggregating joins are lightweight, and are executed using the standard join operator in DuckDB, producing an intermediate table containing both qualifying order keys and the associated customer regions. We focus on the subsequent costly LINEITEM-ORDERS binary join. Selectivity for this operation is controlled by a predicate on  $l\_orderkey$ .

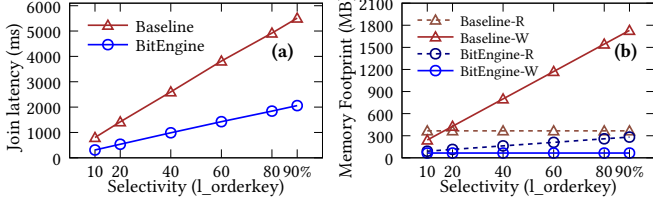
**Baseline.** Most columnar DBMSs employ hash joins and adopt either early or late materialization strategies, each offering distinct tradeoffs [1, 6, 33]. To minimize cache misses during the probe phase, the baseline DBMS materializes the inner table by converting qualifying tuples into row format (§2.2) when building the hash table. In our evaluation, we select LINEITEM as the inner table due to the low selectivity of the  $l\_orderkey$  predicate and the minimal requirement to materialize only two columns:  $l\_price$  and  $l\_disc$ . (In contrast, materializing the ORDERS table incurs a cost of 42 bytes per row, as it must store foreign keys to *CUSTOMER*, *NATION*, and *REGION*, along with the  $n\_name$  string attribute.)

**BITENGINE’s Join operator** transforms a binary join into a predicate on the inner table’s foreign key column. By utilizing a bitmap index, it efficiently retrieves tuples that satisfy the join condition. In our example, a bitmap is maintained for  $l\_orderkey$  (the blue box in Figure 9). Following the baseline, BITENGINE first joins ORDERS with other dimension tables to produce an intermediate table. Based on this, BITENGINE extracts a set of qualifying order keys,  $K$ , organized per region (the red box). Traditionally, the system would compare each value in  $l\_orderkey$  against  $K$  to perform the join. Instead, the  $l\_orderkey$  bitmap index identifies matching records for each order key using corresponding bitvectors. For example, reading the bitvector for value 1 immediately identifies all LINEITEM tuples where  $l\_orderkey$  is 1. Based on this, for each region’s order keys, BITENGINE’s Join operator (1) retrieves the corresponding bitvectors, (2) ORs them to produce a result bitvector representing the region’s qualifying tuples, (3) performs SIMD aggregations on the aggregation columns using this result bitvector for AVX-512 mask registers (§3.3), and (4) populates the output with computed aggregates. For example, because orders 1 and 4 belong to customers in Africa, the bits corresponding to their lineitems are set in the result bitvector, as depicted in Figure 9.

Our Join operator is motivated by *invisible join* [1] but eliminates its most expensive stage (comparing each value in  $l\_orderkey$  against  $K$ ) by utilizing bitmap indexing. Furthermore, BITENGINE eliminates the need to materialize the inner table or perform subsequent out-of-order scans on the outer table [1]. This not only ensures fast join operations but also removes the intermediate memory overhead typically associated with hash joins (see below).

**Observations.** By adjusting the  $l\_orderkey$  predicate to vary selectivity, we make the following observations.





**Figure 10:** By eliminating the costly materialization phase inherent in traditional hash joins, BITENGINE’s Join operator (a) outperforms the baseline across selectivities and (b) produces intermediate data orders of magnitude smaller, making this design highly effective for large-scale datasets.

*BITENGINE Outperforms the Baseline Across Selectivities* because it eliminates the expensive inner table materialization required by standard Join operators. As the selectivity on  $l\_orderkey$  increases, the baseline’s materialization costs scale linearly. While BITENGINE’s costs also rise due to the increased number of bitvectors to be merged, this overhead is largely mitigated by the Group Encoding scheme of the bitmap indexes. As shown in Figure 10a, BITENGINE’s Join operator is 1.6 $\times$  faster than the baseline when the selectivity of  $l\_orderkey$  is 10%, and becomes 2.6 $\times$  faster for 90% selectivity.

*BITENGINE Reads Less Data.* We measure the data reads for different Join implementations. While the baseline reads the join column ( $l\_orderkey$ ), BITENGINE reads the qualifying compressed bitvectors from the bitmap index, which are generally smaller than reading raw data. This efficiency is quantified in Figure 10b (dashed lines labeled -R). As selectivity increases from 10% to 90%, BITENGINE’s Join operator reads 90MB to 282MB of data, representing 25% to 77% of the data read by the baseline.

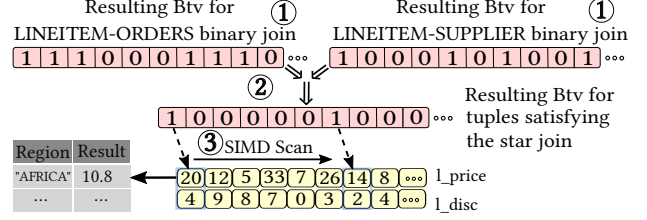
*BITENGINE Generates Negligible Intermediate Data.* BITENGINE’s intermediate data consists of only the keys from preceding operators and a result bitvector (red boxes Figure 9), yielding substantial space savings compared to the baseline. In our example query, BITENGINE generates 65MB of intermediate data, making it 4 $\times$  more space efficient than the baseline that materializes the inner table (§2.2). More importantly, this size is invariant to selectivity on  $l\_orderkey$ , reaching a 27 $\times$  space efficiency at 90% selectivity, as shown in Figure 10b (solid lines labeled -W).

### 3.6 Star Join

Star joins are fundamental to analytical queries. We use the following TPC-H Q5 variant as an example, which extends the query in §3.5 by additionally joining LINEITEM with SUPPLIER.

```
SELECT r_name, SUM(l_price * l_disc)
FROM LINEITEM JOIN ORDERS JOIN SUPPLIER JOIN ...
GROUP BY r_name
```

**Baseline.** Modern DBMSs typically process the query using a sequence of binary joins, where the output of one serves as the input for the next. Ideally, the optimizer orders these joins to minimize intermediate data sizes. However, accurately estimating join cardinality is notoriously difficult [20, 24], as the Vertica team noted, “It is very difficult to determine with any measure of confidence whether a particular join will spill or not, due to inaccuracies in cardinality estimation” [36]. Consequently, star joins often yield unexpectedly large intermediate results, as all binary joins involve LINEITEM.



**Figure 11:** BITENGINE executes a star join via (1) binary joins between fact and dimension tables, producing a bitvector for each. (2) These bitvectors are merged into a final bitvector serving as a mask that identifies qualifying fact table tuples, (3) enabling a SIMD scan on column data.

BITENGINE executes a star join by executing independent binary joins in parallel. Each binary join is transformed into predicates on a foreign key column of LINEITEM, and outputs a bitvector indicating the qualifying tuples (§3.5). These result bitvectors are then merged (ANDed) together, generating a final result bitvector representing tuples that pass the star join. In our example query, BITENGINE’s Join operator (1) binary-joins LINEITEM with ORDERS and SUPPLIER, producing two result bitvectors, (2) efficiently merges these bitvectors, yielding a final bitvector that identifies tuples satisfying both binary joins, and (3) performs SIMD scans on the aggregation columns by loading the final bitvector into AVX-512 mask registers, as illustrated in Figure 11. Our evaluation shows that BITENGINE achieves a 2 $\times$  speedup over the baseline for Q5, while reducing intermediate data by one order of magnitude.

### 3.7 Other Operators

BITENGINE also shows promise in accelerating operators derived from the basic ones described above.

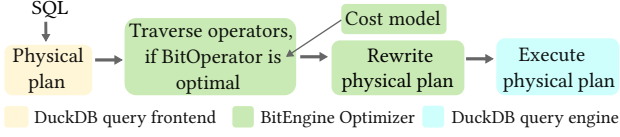
```
SELECT SUM(...) FROM LINEITEM WHERE l_shipmode in
["AIR", ...] and l_shipinstruct in ["DROP SHIP", ...];
```

**String Matching Filters** can benefit substantially from maintaining dictionaries for string attributes and bitmap indexing the encoded reference arrays, an approach similar to DuckDB’s Dictionary Encoding combined with FSST [39]. For example, in the above Q19, we build bitmap indexes for the  $l\_shipmode$  and  $l\_shipinstruct$  attributes, such that the query engine leverages the indexes and consults the dictionaries when necessary, making the BITENGINE 1.5 $\times$  faster than the baseline implementation.

## 4 BITENGINE ON ANALYTICAL DATABASES

We now discuss the integration of BITENGINE into modern analytical databases (§2.2), using DuckDB as a representative example.

**Overall Workflow.** As illustrated in Figure 12, BITENGINE reuses the standard query processing procedure in DuckDB (i.e., parser, logical planner, optimizer, and physical plan generation). Given a pipelined physical plan, BITENGINE traverses the operators bottom-up (from the table scan to the final output) and evaluates whether they can be replaced with BitOperators. Specifically, it checks if bitmap indexes exist for the relevant attributes and whether the bitmap-based operators yield a lower estimated cost (see below). If these conditions are met, BITENGINE updates the physical plan to use BitOperators. All other operators remain unchanged and execute using the native DuckDB implementations.



**Figure 12: BITENGINE optimizes DuckDB physical plans by traversing pipelines bottom-up and substituting BitOperators where optimal. This preserves the underlying execution procedure, ensuring seamless integration into DuckDB.**

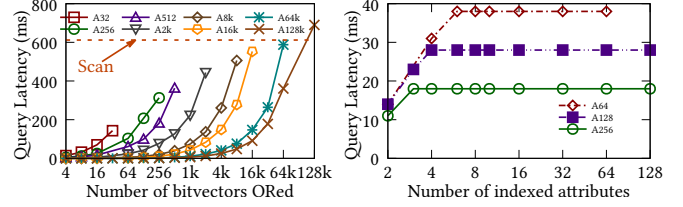
**Cost Model.** Our current cost model is operator-specific and based on the observations from the BitOperators microbenchmarks (§3).

Specifically, BITENGINE always replaces with BitOperators for (1) *Selection* as it consistently reduces bytes read and provides fast queries insensitive to column cardinality and predicate selectivity (§3.1), (2) *Projection*, if the upstream selection operator provides a result bitvector, as it allows for a precise selective read of the projection columns (§3.2), and (3) *Aggregation* as it consistently outperforms alternatives across all selectivities. Furthermore, if the estimated selectivity falls below 4%, BITENGINE converts the bitvector into RIDs, enabling aggregation via probing (§3.3). For *Grouping*, BITENGINE chooses BITENGINE-Probe, BITENGINE-SIMD, and DuckDB’s standard grouping operators when the estimated number of groups is below 4, between 4 and 32, and above 32, respectively (§3.4). These thresholds are empirically determined based on our evaluation results. For *Binary Join*, BITENGINE chooses the BitOperator when the join is followed by Grouping and Aggregation, which is the common case in analytical queries. For non-aggregating joins (e.g., Q2 and Q20), which typically join dimension tables and do not have performance choke points, BITENGINE uses the standard DuckDB join operators. BITENGINE always replaces *Star Joins* with BitOperators, as they outperform standard sequential join pipelines in both memory footprint and query speed (§3.6).

**Replacing Operators.** The Selection, Aggregation, and Grouping BitOperators first compute the result bitvectors and then execute as discussed in §3. Result bitvectors are stored in the query context (termed *executor* in DuckDB) for use by subsequent operators and pipelines. For Binary Join, BITENGINE replaces the build phase by computing qualifying ID lists from the dimension table and replaces the probe phase by retrieving bitvectors from the bitmap instance on the foreign key attribute of the fact table. In the dominant case (see §3.5), where a Join operator is followed by Grouping and Aggregation, BITENGINE fuses these pipelines, by performing grouping directly on the qualifying ID lists from the dimension tables before executing the Join. For Star Joins, BITENGINE replaces sequential binary join pipelines in DuckDB with independent binary joins that produce result bitvectors, which are then merged.

## 5 BITENGINE IMPLEMENTATION

**Efficient Bitvector Merges.** BITENGINE’s Join operator needs to retrieve a large number of bitvectors and perform extensive bitwise operations over them, when applied to high-cardinality attributes. For example, in our simplified TPC-H Q5 example, when  $SF = 10$ , the bitmap instance on the  $L\_orderkey$  attribute of the *LINEITEM* table contains ~15 million bitvectors, and our Join operator involves ~357K logical ORs, previously a prohibitive cost. We address this inefficiency effectively from the following perspective.



**Figure 13: (a) ORing 64K or fewer bitmap bitvectors is faster than scanning the corresponding column data, and (b) the cost of ANDing bitvectors from different bitmap instances increases for a few ANDs and then flattens.**

**BITENGINE’s Merge Mechanism** leverages the observation that, for high-cardinality attributes using the EE or GE schemes, each bitvector contains a few 1s (~4 1s in this bitmap instance) and is highly compressible (~16 bytes/bitvector), as shown in Figure 9. This allows the merge mechanism to skip most portions of the bitvector during merging. To demonstrate this, we evaluate the cost of merging a group of bitvectors. Figure 13a presents the execution time for bitwise ORs across a group of bitvectors while varying the cardinality of the indexed attributes from 32 to 128K (as indicated by the legend suffixes). For comparison, we also include execution time for scanning the column using a two-sided predicate. We observe that ORing 64K bitvectors to retrieve the result bitvector is faster than scanning the base data. The main reasons are as follows. (1) With higher cardinality, the bitvectors have lower bit density and are more compressible, allowing us to skip most portions of the intermediate bitvector (*IB*) during merging. (2) As 1s accumulate, *IB* becomes less compressible and is maintained as decompressed, significantly reducing the cost of merging 1s into it.

**Multi-Column Predicates.** Realistic queries often involve logical ANDs over bitvectors generated by multi-column predicates and star joins. For example, in TPC-H Q5, the final result bitvector for  $L\_price$  is computed by ANDing the join result bitvectors from *LINEITEM-ORDERS* and *LINEITEM-SUPPLIER* (§3.6). Figure 13b reports the execution time for logical ANDs between bitvectors from different bitmap instances, with cardinality varying from 64 to 256. Our evaluation shows that the query latency plateaus after a few ANDs because *IB*’s bit density decreases dramatically and becomes highly compressible, enabling the skipping of most portions in the subsequent ANDs, making ANDing join result bitvectors efficient.

**Group Encoding Reduces Bitvectors Merged.** With GE, when most bitvectors in a group are selected, we use the cumulative bitvector and subtract the non-matching ones. This process can be accelerated by decreasing the GE group size, at the expense of an increased index size.

**Bitmap Organization.** For high-cardinality attributes, BITENGINE compactly stores each group of bitvectors together to reduce the metadata overhead of managing millions of bitvectors and to improve cache locality. For example, in TPC-H workloads, BITENGINE stores 6K bitvectors (16 bytes each) for  $L\_orderkey$  in a single file.

**Non-Destructive SIMD Aggregation.** BitOperators often aggregate over the same columns iteratively, using different bitvectors as masks. For example, in grouping (§3.4), aggregation columns are scanned multiple times, each time using a different group-by result bitvector. Legacy SIMD implementations (e.g., SSE) must repeatedly

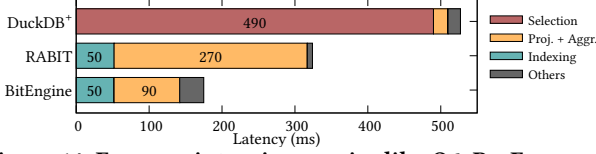


Figure 14: For scan-intensive queries like Q6, BITENGINE reduces bytes read for selection and smartly adapts subsequent projection and aggregation strategies based on selection results, significantly reducing the selection time and the subsequent projection/aggregation time compared to baselines.

scan the column data for each bitvector because they rely on destructive instructions that overwrite input registers. BITENGINE solves this inefficiency by leveraging *non-destructive* AVX-512 instructions that write results to separate registers. This allows the column data to be preserved in the source `__m512i` registers across multiple aggregations, resulting in significantly fewer capacity cache misses. We apply this optimization to both Grouping and Join operators.

## 6 EXPERIMENTAL EVALUATION

We now show that BITENGINE enables bitmap indexing in modern analytical DBMSs, providing superior query performance.

### 6.1 Methodology

All experiments are conducted on a server with two Intel Xeon 8570 CPUs (each with 56 cores and 300MB shared L3 cache), 192GB DDR5 DRAM, and a 1TB SSD. We also evaluate BITENGINE on another server with a recent AMD EPYC 9965 processor with native AVX-512 support to demonstrate the generality of our SIMD optimizations without requiring additional engineering efforts. As results on both servers exhibit similar performance trends, we omit the EPYC data for brevity. We evaluate our system using the TPC-H benchmark with scale factor SF = 10. BITENGINE and the baselines are compiled using g++ 13.2 with -O3 on Ubuntu 24.04. For performance tests, we run each query 1,000 times and report the mean values. Standard deviations > 2% are denoted by error bars.

### 6.2 BITENGINE Benefits Columnar DBMSs

We integrate BITENGINE into DuckDB [32], an open-source analytical system with a sophisticated query engine. We also integrate and compare with RABIT [44], state-of-the-art bitmap index design, to demonstrate the benefits of BITENGINE’s overall design.

**DuckDB+.** We use the official DuckDB v1.3 as our baseline. While this version is substantially more sophisticated and efficient than its initial releases [32], its query optimizer still misses certain TPC-H optimizations. For example, Q10 performs grouping on attributes `c_custkey`, `c_name`, and `c_acctbal`. Since `c_custkey` is the primary key, an optimized plan could defer loading the dependent attributes until after the joins to significantly reduce materialization overhead, an optimization DuckDB currently lacks. To ensure a fair comparison, we manually addressed these known bottlenecks [9, 13], demonstrating that BITENGINE outperforms state-of-the-art columnar systems even against an optimized baseline, denoted as DuckDB+. **RABIT** functions as a standalone index structure that mainly supports point and range queries on a single column, which creates limitations when integrated with DuckDB. Specifically, RABIT (1) lacks selective retrieval of aggregation columns based on prior selection results (§3.2), (2) does not adapt aggregation (§3.3) or grouping

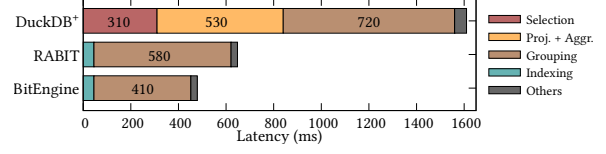


Figure 15: For grouping-heavy queries like Q1, BITENGINE avoids the maintenance of traditional intermediate grouping tables and accelerates aggregations with AVX-512 instructions, significantly reducing query latency.

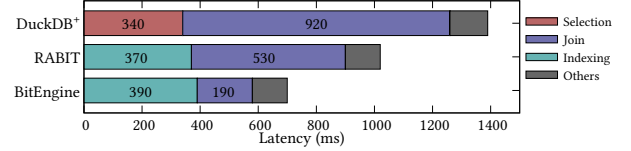


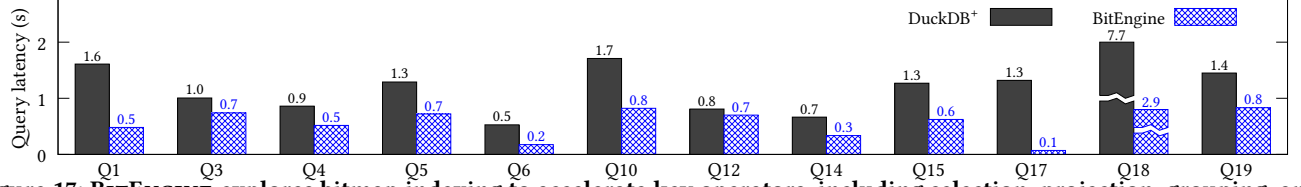
Figure 16: For join-dominated queries like Q5, BITENGINE transforms joins into bitmap-based predicates, significantly reducing the join time by leveraging bitmap indexes.

(§3.4) strategies based on query selectivity, and (3) does not natively support multi-column predicates (§3.1) and star joins (§3.6).

**BITENGINE** enables bitmap indexing even on frequently updated, high-cardinality attributes, allowing us to index attributes in the LINEITEM fact table ranging from the low-cardinality `l_quantity` (cardinality=50) to the high-cardinality `l_orderkey` (cardinality=15M when SF=10). In evaluation, these indexes are built before executing the queries and materialized on disk. For each query, if the involved columns are indexed, BITENGINE attempts to use BitOperators according to the cost model described in §4 to optimize the query execution; otherwise, it falls back to the original DuckDB operators. **Benefits.** To demonstrate the benefits of BITENGINE, we detail the performance of three representative queries: TPC-H Q6 (scan-intensive), Q1 (aggregation-heavy), and Q5 (join-dominated).

**Fast Selection with BITENGINE.** Selection operations dominate analytical workloads. In Q6, DuckDB+ reads the four relevant LINEITEM columns (`l_quantity`, `l_shipdate`, `l_discount`, and `l_extendedprice`) into Data Chunks, and updates SELs to filter data. This operator takes 490ms, accounting for 93% of the total query latency, as shown in Figure 14. The subsequent projection and aggregation operators together take 20ms because Q6 has a low selectivity (2%), skipping 98% of the data. RABIT employs bitmap indexing on predicate columns (`l_shipdate`, `l_discount`, and `l_quantity`), with group sizes set, respectively, to 3 months, `l_discount` cardinality (effectively reducing to EE encoding), and 5, accommodating the requirements of most queries, including Q6. RABIT’s selection operator retrieves the relevant bitvectors and performs 11 logical OR/ANDs to compute the final selection bitvector, avoiding scanning `l_quantity` and `l_shipdate` and reducing the selection time to 50ms. It then performs a Btv-SIMD aggregation (§3.3) on the aggregation columns (`l_discount` and `l_extendedprice`), which takes 270ms. BITENGINE further optimizes the query process by (1) selectively retrieving only the qualifying segments of the projection columns (§3.2), and (2) converting the result selection bitvector to RIDs for probing on aggregation columns (Btv-RID-Probe in §3.3), cutting the projection and aggregation time down to 90ms. Overall, BITENGINE achieves a total query latency of 155ms for Q6, which is 3.0× faster than the optimized DuckDB+ and 1.9× faster than RABIT.





**Figure 17: BITENGINE explores bitmap indexing to accelerate key operators, including selection, projection, grouping, and joins. When integrated into an industry-grade DBMS, BITENGINE matches or outperforms baselines across all TPC-H queries.**

*Fast Grouping with BITENGINE.* Grouping is expensive because frequent updates to intermediate aggregation tables often lead to cache misses. As illustrated in Figure 15, although DuckDB+ employs a perfect hash-based grouping strategy that ensures no hash collisions, grouping in Q1 still takes 720ms, accounting for 45% of the total query latency. Furthermore, since DuckDB+ passes RIDs between operators, it reads all relevant columns into Data Chunks and performs scalar-based aggregation (RID-Scalar in §3.3), which leads to significant overhead in both column reading (310ms) and aggregation (530ms) because Q1 has a high selectivity of 98% and processes almost all the data. RABIT leverages bitmap indexing on the group-by and predicate attributes  $L_{returnflag}$  (card. = 3),  $L_{linestatus}$  (card. = 2), and  $L_{shipdate}$  (card. = 2,526 and GE group size = 3 months), and spends only 45ms for indexing (i.e., computing result bitvectors for each group), omitting reading these columns. It then performs Btv-SIMD aggregation for each group, which takes 580ms. BITENGINE further optimizes this process by traversing these aggregation columns once using non-destructive SIMD aggregation (§5) to update the aggregation results for all groups, significantly reducing cache misses. Overall, BITENGINE achieves a total latency of 480ms for Q1, 3.4× and 1.3× faster than the optimized DuckDB+ and RABIT, respectively.

*Fast Join with BITENGINE.* Join operators are expensive, particularly when fact tables are involved. In Q5, DuckDB+ spends 340ms reading the join columns from the LINEITEM and ORDERS tables and 920ms executing the hash join. As shown in Figure 16, this accounts for 84% of the total query latency. RABIT transforms the LINEITEM-ORDERS join into a predicate on  $L_{orderkey}$  (cardinality = 15M), which is efficiently handled by bitmap indexing by retrieving the matching bitvectors and performing logical ORs between them (§3.5). When SF = 10, this process involves ~357K logical ORs, which are efficiently handled by BITENGINE’s merging mechanism (§5) in 370ms. The result bitvector is subsequently used to retrieve the qualifying tuples from the LINEITEM table, which are then joined with the SUPPLIER table using a standard hash join, taking 530ms. BITENGINE further optimizes this process by additionally converting the LINEITEM-SUPPLIER join into a predicate on  $L_{suppkey}$  (§3.6) with cardinality = 10K, which adds 20ms to merge the related bitvectors. The result bitvectors from the two binary joins are then combined using a bitwise AND. This combined bitvector achieves a much lower selectivity (0.1%) than the initial LINEITEM-ORDERS join (2.4%), allowing BITENGINE to convert it to RIDs and probe the LINEITEM table (Btv-RID-Probe in §3.3), significantly reducing the subsequent scan and aggregation time to 190ms. Overall, by eliminating the overhead of intermediate hash tables, BITENGINE achieves a Q5 latency of 720ms, which is 1.4× faster than RABIT and 1.8× faster than the optimized DuckDB+.

**Full TPC-H Evaluation.** We evaluate BITENGINE on all TPC-H queries. Our results demonstrate that BITENGINE substantially

improves 12 out of 22 TPC-H queries with inherent performance choke points, covering join-dominated (Q4, Q5, Q17, and Q19), aggregation-heavy (Q1 and Q18), and scan-intensive (Q6 and Q12) queries. As shown in Figure 17, BITENGINE yields a speedup of 1.4×–18.9× compared to DuckDB+. On the remaining queries (e.g., Q2 and Q16) that lack significant performance bottlenecks, BITENGINE matches the baseline performance of DuckDB+.

## 7 RELATED WORK

We classify research on bitmap-based indexes as follows.

**Bitmap Indexes in DBMSs.** While some commercial DBMSs (e.g., Oracle) support bitmap indexes [28, 30], their designs are proprietary. Broadly, the database community has been reluctant to adopt bitmap indexes, in part due to limited open knowledge and research. For example, PostgreSQL introduced a native bitmap index in version 8.3.23, but later removed it due to its limitations (low-cardinality attributes under read-mostly workloads) and limited applicability (accelerating scans only). BITENGINE is the first systematic research on exploring broader use cases for bitmap indexes.

**Bitmap-Like Structures** are widely used in operators because of their compact memory footprint and query speed. For example, PostgreSQL implements a Bloom filter index [14] to quickly exclude non-matching tuples via bit-string signatures, though it supports only equality predicates, and prior work [8] uses bitmaps to significantly reduce the memory footprint of hash joins. In contrast, our bitmaps serve as general-purpose secondary indexes.

**Join Algorithms.** A recent study shows that non-partitioned hash joins are more efficient than sort-merge joins for uniformly distributed workloads [5, 18] and avoid the precomputation overhead associated with radix hash joins [7, 33]. Furthermore, it is well studied that hash joins using late materialization (i.e., storing only the dimension-table keys in the hash table and fetching payload columns by scanning the dimension table after the join operation) results in out-of-order probing and degraded join performance [23].

## 8 CONCLUSION

While recent advancements have addressed the fundamental shortcomings of bitmap indexes (lack of updatability and support for high-cardinality attributes), their utility beyond simple scan accelerators in DBMSs remains an open question. We present BITENGINE, a systematic redesign of an analytical query engine around bitmap indexing. By designing fundamental physical operators (including selection, projection, aggregation, grouping, and joins) that place modern bitmap indexes at their core, BITENGINE demonstrates notable benefits compared to the baselines. These advantages include reduced I/Os and intermediate data, improved SIMD utilization, and avoidance of costly materialization. Evaluation shows that BITENGINE significantly accelerates analytical queries.



## REFERENCES

- [1] Daniel J Abadi, Samuel Madden, and Nabil Hachem. 2008. Column-stores vs. row-stores: how different are they really?. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 967–980. <https://doi.org/10.1145/1376616.1376712>
- [2] Daniel J. Abadi, Daniel S. Myers, David J. DeWitt, and Samuel R. Madden. 2007. Materialization Strategies in a Column-Oriented DBMS. In *Proceedings of the IEEE International Conference on Data Engineering (ICDE)*, 466–475. [http://ieeexplore.ieee.org/xpls/abs\\_all.jsp?arnumber=4221695](http://ieeexplore.ieee.org/xpls/abs_all.jsp?arnumber=4221695)
- [3] Apache. 2023. Parquet. <https://parquet.apache.org/> (2023).
- [4] Manos Athanassoulis, Zheng Yan, and Stratos Idreos. 2016. UpBit: Scalable In-Memory Updatable Bitmap Indexing. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 1319–1332. <https://dl.acm.org/citation.cfm?id=2915964>
- [5] Cagri Balkesen, Gustavo Alonso, Jens Teubner, and M Tamer Özsu. 2013. Multi-Core, Main-Memory Joins: Sort vs. Hash Revisited. *Proceedings of the VLDB Endowment* 7, 1 (2013), 85–96. <http://www.vldb.org/pvldb/vol7/p85-balkesen.pdf>
- [6] Cagri Balkesen, Jens Teubner, Gustavo Alonso, and M. Tamer Özsu. 2013. Main-memory hash joins on multi-core CPUs: Tuning to the underlying hardware. In *Proceedings of the IEEE International Conference on Data Engineering (ICDE)*. 362–373. <http://dl.acm.org/citation.cfm?id=2510649.2511320>
- [7] Maximilian Bandle, Jana Giceva, and Thomas Neumann. 2021. To Partition, or Not to Partition, That is the Join Question in a Real System. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 168–180. <https://doi.org/10.1145/3448016.3452831>
- [8] Ronald Barber, Guy M Lohman, Ippokratis Pandis, Vijayshankar Raman, Richard Sidle, Gopi K Attaluri, Naresh Chainani, Sam Lightstone, and David Sharpe. 2014. Memory-Efficient Hash Joins. *Proceedings of the VLDB Endowment* 8, 4 (2014), 353–364. <http://www.vldb.org/pvldb/vol8/p353-barber.pdf>
- [9] Peter A. Boncz, Thomas Neumann, and Orri Erling. 2013. TPC-H Analyzed: Hidden Messages and Lessons Learned from an Influential Benchmark. In *Proceedings of the TPC Technology Conference on Performance Evaluation, Measurement and Characterization of Complex Systems (TPCTC)*. [https://link.springer.com/chapter/10.1007/978-3-319-04936-6\\_5](https://link.springer.com/chapter/10.1007/978-3-319-04936-6_5)
- [10] Peter A. Boncz, Marcin Zukowski, and Niels J. Nes. 2005. MonetDB/X100: Hyper-Pipelining Query Execution. In *Proceedings of the Biennial Conference on Innovative Data Systems Research (CIDR)*.
- [11] Chee-Yong Chan and Yannis E. Ioannidis. 1998. Bitmap index design and evaluation. *ACM SIGMOD Record* 27, 2 (1998), 355–366. <http://dl.acm.org/citation.cfm?id=276305.276336>
- [12] Chee-Yong Chan and Yannis E. Ioannidis. 1999. An efficient bitmap encoding scheme for selection queries. *ACM SIGMOD Record* 28, 2 (1999), 215–226. <http://dl.acm.org/citation.cfm?id=304181.304201>
- [13] Markus Dreseler, Martin Boissier, Tilmann Rabl, and Matthias Uflacker. 2020. Quantifying TPC-H Choke Points and Their Optimizations. *Proceedings of the VLDB Endowment* 13, 8 (2020), 1206–1220. <http://www.vldb.org/pvldb/vol13/p1206-dreseler.pdf>
- [14] PostgreSQL Global Development Group. 2016. Bloom Filter Index Access Method. <https://www.postgresql.org/docs/current/bloom.html>. [Online; accessed 1-Feb-2025].
- [15] John L. Hennessy and David A. Patterson. 2012. *Computer Architecture - A Quantitative Approach, 5th Edition*. Morgan Kaufmann.
- [16] Brian Hentschel, Michael S Kester, and Stratos Idreos. 2018. Column Sketches: A Scan Accelerator for Rapid and Robust Predicate Evaluation. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 857–872. <http://doi.acm.org/10.1145/3183713.3196911>
- [17] Michael S. Kester, Manos Athanassoulis, and Stratos Idreos. 2017. Access Path Selection in Main-Memory Optimized Data Systems: Should I Scan or Should I Probe?. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 715–730. <http://dl.acm.org/citation.cfm?doid=3035918.3064049>
- [18] Changkyu Kim, Eric Sedlar, Jatin Chhugani, Tim Kaldewey, Anthony D Nguyen, Andrea Di Blas, Victor W Lee, Nadathur Satish, and Pradeep Dubey. 2009. Sort vs. Hash Revisited: Fast Join Implementation on Modern Multi-Core CPUs. *Proceedings of the VLDB Endowment* 2, 2 (2009), 1378–1389. <http://www.vldb.org/pvldb/vol2/vldb09-257.pdf>
- [19] Harald Lang, Alexander Beischl, Viktor Leis, Peter A Boncz, Thomas Neumann, and Alfons Kemper. 2020. Tree-Encoded Bitmaps. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 937–967. <https://doi.org/10.1145/3318464.3380588>
- [20] Viktor Leis, Andrey Gubichev, Atanas Mirchev, Peter A Boncz, Alfons Kemper, and Thomas Neumann. 2015. How Good Are Query Optimizers, Really? *Proceedings of the VLDB Endowment* 9, 3 (2015), 204–215. <http://www.vldb.org/pvldb/vol9/p204-leis.pdf>
- [21] Linwei Li, Kai Zhang, Jiading Guo, Wen He, Zhenying He, Yinan Jing, Weili Han, and X Sean Wang. 2020. BinDex: A Two-Layered Index for Fast and Robust Scans. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 909–923. <https://doi.org/10.1145/3318464.3380563>
- [22] Yinan Li and Jignesh M Patel. 2013. BitWeaving: Fast Scans for Main Memory Data Processing. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 289–300. <http://doi.acm.org/10.1145/2463676.2465322>
- [23] Yihao Liu, Shaoxuan Tang, Yulong Hui, Hangrui Zhou, and Huanchen Zhang. 2025. Selective Late Materialization in Modern Analytical Databases. *Proc. VLDB Endow.* 18, 11 (2025). <https://doi.org/10.14778/3749646.3749717>
- [24] Guy M. Lohman. 2014. Is Query Optimization a "Solved" Problem? *ACM SIGMOD Blog* (2014). <https://wp.sigmod.org/?p=1075>
- [25] Hung Q. Ngo, Ely Porat, Christopher Ré, and Atri Rudra. 2018. Worst-case Optimal Join Algorithms. *J. ACM* 65, 3 (2018), 40 pages. <https://doi.org/10.1145/3180143>
- [26] Patrick E. O’Neil. 1987. Model 204 Architecture and Performance. In *Proceedings of the International Workshop on High Performance Transaction Systems (HPTS)*. 40–59. <http://dl.acm.org/citation.cfm?id=645575.658338>
- [27] Patrick E. O’Neil and Dallan Quass. 1997. Improved query performance with variant indexes. *ACM SIGMOD Record* 26, 2 (1997), 38–49. <http://dl.acm.org/citation.cfm?id=253262.253268>
- [28] Oracle. 2013. Oracle11g Database Data Warehousing Guide. [https://docs.oracle.com/cd/E11882\\_01/server.112/e25554/indexes.htm](https://docs.oracle.com/cd/E11882_01/server.112/e25554/indexes.htm). [Online; accessed 1-May-2024].
- [29] Oracle. 2024. Oracle23ai Data Warehousing Guide. <https://docs.oracle.com/en/database/oracle/oracle-database/23/dwhsg/index.html>. [Online; accessed 1-October-2024].
- [30] Oracle. 2024. Oracle23ai Database Concepts. <https://docs.oracle.com/en/database/oracle/oracle-database/23/cncpt/indexes-and-index-organized-tables.html#GUID-B15C4817-7748-456D-9740-8B9628AF9F47>. [Online; accessed 1-October-2024].
- [31] Orestis Polychroniou, Arun Raghavan, and Kenneth A. Ross. 2015. Rethinking SIMD Vectorization for In-Memory Databases. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 1493–1508. <http://doi.acm.org/10.1145/2723372.2747645>
- [32] Mark Raasveldt and Hannes Mühleisen. 2019. DuckDB: An Embeddable Analytical Database. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 1981–1984. <https://doi.org/10.1145/3299869.3320212>
- [33] Stefan Schuh, Xiao Chen, and Jens Dittrich. 2016. An Experimental Comparison of Thirteen Relational Equi-Joins in Main Memory. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 1961–1976. <https://doi.org/10.1145/2882903.2882917>
- [34] Robert Schulze, Tom Schreiber, Ilya Yatsishin, Ryadh Dahimene, and Alexey Milovidov. 2024. ClickHouse - Lightning Fast Analytics for Everyone. *Proc. VLDB Endow.* 17, 12 (2024). <https://doi.org/10.14778/3685800.3685802>
- [35] Patricia G. Selinger, Morton M. Astrahan, Donald D. Chamberlin, Raymond A. Lorie, and Thomas G. Price. 1979. Access Path Selection in a Relational Database Management System. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 23–34. <http://dl.acm.org/citation.cfm?id=582095.582099>
- [36] Lakshmi Kant Shrinivas, Sreenath Bodagala, Ramakrishna Varadarajan, Ariel Cary, Vivek Bharathan, and Chuck Bear. 2013. Materialization strategies in the Vertica analytic database: Lessons learned. In *2013 IEEE 29th International Conference on Data Engineering (ICDE)*. 1196–1207. <https://doi.org/10.1109/ICDE.2013.6544909>
- [37] Lefteris Sidiropoulos and Martin L. Kersten. 2013. Column Imprints: A Secondary Index Structure. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 893–904. <http://dl.acm.org/citation.cfm?id=2463676.2465306>
- [38] Avi Silberschatz, Henry F Korth, and S Sudarshan. 2020. *Database System Concepts, Seventh Edition*. McGraw-Hill Book Company. <https://www.db-book.com/DuckDBTeam>
- [39] DuckDB Team. 2022. Lightweight Compression in DuckDB. <https://duckdb.org/2022/10/28/lightweight-compression.html>. [Online; accessed 1-May-2024].
- [40] Sybase IQ Team. 2009. Chapter 15.1: Overview of Indexes. <https://infocenter.sybase.com/help/index.jsp?topic=/com.sybase.infocenter.dc00170.1510/html/igapgv1/CIHECEAI.htm>. [Online; accessed 1-October-2024].
- [41] Midhul Vuppapapati, Justin Miron, Rachit Agarwal, Dan Truong, Ashish Motivala, and Thierry Cruanes. 2020. Building an elastic query engine on disaggregated storage. In *Proceedings of the 17th Usenix Conference on Networked Systems Design and Implementation*. 449–462.
- [42] Junchang Wang and Manos Athanassoulis. 2024. CUBIT: Concurrent Updatable Bitmap Indexing. *Proceedings of the VLDB Endowment* 18, 2 (2024), 399–412. <https://www.vldb.org/pvldb/vol18/p399-athanassoulis.pdf>
- [43] Junchang Wang and Manos Athanassoulis. 2024. CUBIT: Concurrent Updatable Bitmap Indexing (Extended Version). *CoRR* 2410.16929 (2024). <https://arxiv.org/abs/2410.16929>
- [44] Junchang Wang, Fu Xiao, and Manos Athanassoulis. 2025. RABIT: Efficient Range Queries with Bitmap Indexing. *Proceedings of the ACM on Management of Data (PACMOD)* 3, 6 (2025), to appear.
- [45] Thomas Willhalm, Nicolae Popovici, Yazan Boshmaf, Hasso Plattner, Alexander Zeier, and Jan Schaffner. 2009. SIMD-Scan: Ultra Fast in-Memory Table Scan using on-Chip Vector Processing Units. *Proceedings of the VLDB Endowment* 2, 1 (2009), 385–394. <http://www.vldb.org/pvldb/vol2/vldb09-327.pdf>
- [46] Harry K T Wong, Hsiu-Fen Liu, Frank Olken, Doron Rotem, and Linda Wong. 1985. Bit Transposed Files. In *Proceedings of the International Conference on Very*

- Large Data Bases (VLDB)*. 448–457. <http://www.vldb.org/conf/1985/P448.PDF>
- [47] Kesheng Wu, S Ahern, E W Bethel, J Chen, H Childs, E Cormier-Michel, C Geddes, J Gu, H Hagen, B Hamann, W Kogler, J Lauret, J Meredith, P Messmer, Ekow J. Otoo, V Perevotzhikov, A Poskanzer, O Rübel, Arie Shoshani, A Sim, Kurt Stockinger, G Weber, and W-M Zhang. 2009. FastBit: interactively searching massive data. *Journal of Physics: Conference Series* 180, 1 (2009), 012053. <http://iopscience.iop.org/1742-6596/180/1/012053>
- [48] Kesheng Wu, Ekow J. Otoo, and Arie Shoshani. 2002. Compressing Bitmap Indexes for Faster Search Operations. In *Proceedings of the 14th International Conference on Scientific and Statistical Database Management*. 99–108. <https://doi.org/10.1109/SSDM.2002.1029710>